



PROGRAMA DE FORMACIÓN MARVELL

Diseño Digital para Sistemas DSP

Ejercicios Módulo 6: Planificación Temporal

Profesor: Del Barco, Martín.

Alumnos: Monja, Ernesto Joaquín
Zúñiga, Guillermo Rubén Darío

Índice

1. Introducción	2
2. Desarrollo	2
2.1. Data Flow Graph de un <i>FIR</i>	2
2.2. <i>Scheduling ASAP</i> y <i>ALAP</i>	3
2.3. Versión Iterativa con <i>MAC</i>	4
2.4. <i>Feed-Forward Cut-Set</i>	5
2.5. Comparación del <i>PPA</i>	6
2.6. Límite de Iteración (<i>IPB</i>)	7
3. Conclusiones	8
4. Referencias	10

1. Introducción

El presente informe tiene como objetivo analizar y aplicar los conceptos de planificación temporal sobre la arquitectura de un filtro FIR directo de 4 coeficientes. Para ello, se empleó la representación formal mediante *Grafos de Flujo de Datos* (DFG), identificando nodos, aristas y elementos de retardo (registros). Sobre este modelo, se aplicaron técnicas de *scheduling* y se exploraron dos estrategias contrapuestas: una implementación iterativa basada en el reuso de recursos y el método de *pipelining* para minimizar el camino crítico. Todo esto para evaluar y comparar el *PPA* entre ambas soluciones. Finalmente, se evalúa el *límite de iteración* (IPB) introducido por la realimentación en una topología IIR, y cómo varía al aplicar la técnica de *C-Slow*.

2. Desarrollo

2.1. Data Flow Graph de un *FIR*

Un filtro *FIR* de 4 coeficientes (taps) en forma directa se describe matemáticamente mediante la ecuación de diferencias presentada en la [Ec. 1](#).

$$y[n] = h_0x[n] + h_1x[n-1] + h_2x[n-2] + h_3x[n-3] \quad (1)$$

En la [Fig. 1](#) podemos apreciar la estructura en forma de *DFG* (Data Flow Graph).

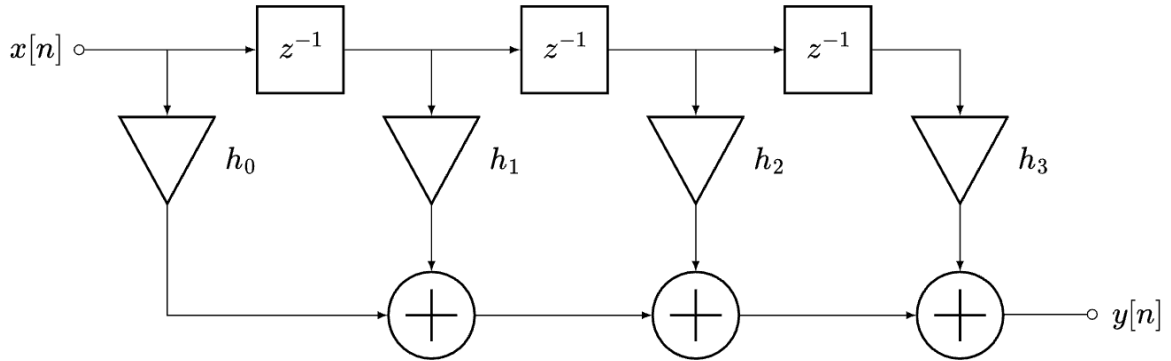


Figura 1: *DFG* de un filtro *FIR*. [\[1\]](#)

En esta figura podemos apreciar los siguientes elementos del *DFG*:

- Nodos (Operaciones): El grafo contiene 4 multiplicadores (correspondientes a la multiplicación por los coeficientes h_0, h_1, h_2 y h_3) y 3 sumadores encargados de acumular los productos.
- Aristas (Dependencias): Son las conexiones que dirigen los datos desde la señal de entrada $x[n]$ hacia los multiplicadores, y las salidas de estos hacia la cadena de

sumadores.

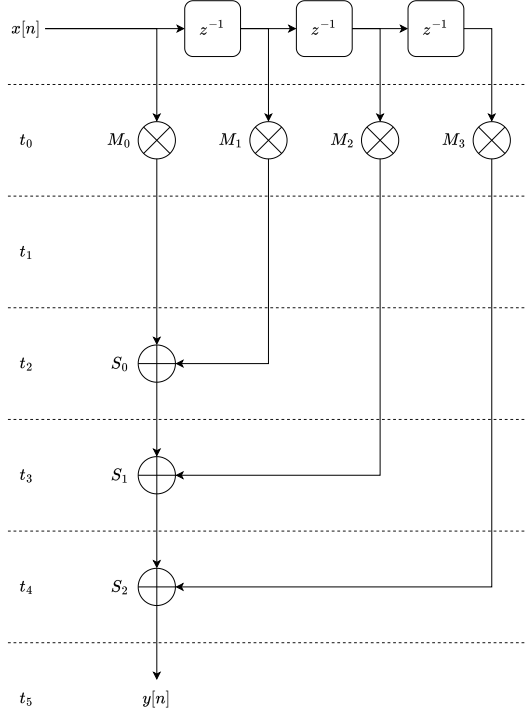
- Registros (z^{-1}): Se necesitan 3 registros de retardo ubicados secuencialmente en la línea de entrada para almacenar los valores históricos $x[n-1]$, $x[n-2]$ y $x[n-3]$.

2.2. *Scheduling ASAP y ALAP*

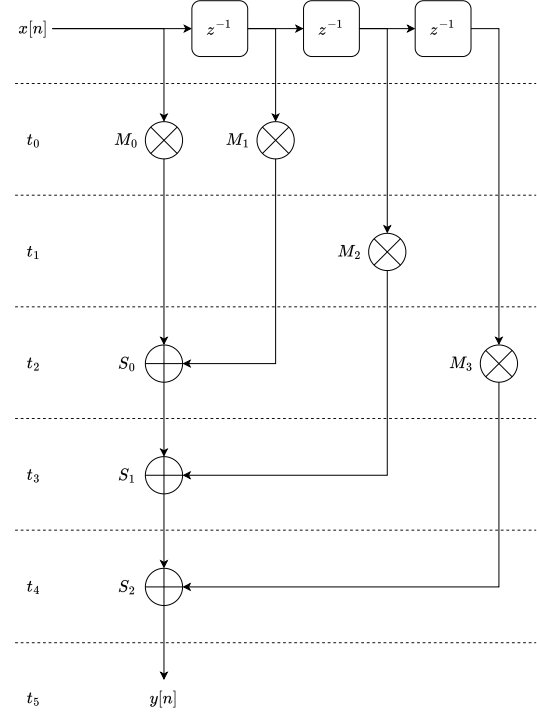
El método de *scheduling* consiste en definir los instantes en los que cada operación de un grafo se debería ejecutar, respetando dos tipos de restricciones: de dependencia, donde una operación solo puede ejecutarse cuando sus operandos estén disponibles; y de recursos, donde el número de sumadores, multiplicadores, etc. es finito. Dentro de este método existen dos estrategias:

- ASAP (As Soon As Possible): Cada operación se ejecuta lo antes posible. De aquí se obtiene la latencia total del circuito.
- ALAP (As Late As Possible): Cada operación se ejecuta lo más tarde posible, pero sin afectar la latencia total. En este caso se descubre la flexibilidad temporal de cada operación

Con estos dos enfoques, se realizó el *scheduling* del filtro *FIR* de la [Subsección 2.1](#). Este caso se muestra en la [Fig. 2](#), donde se propone que un sumador tiene una latencia de 1 ut (1 unidad de tiempo), mientras que un multiplicador tiene una de 2 ut . El *scheduling* *ASAP* se encuentra en la [Fig. 2a](#), y se observa que el camino crítico (1 multiplicador y 3 sumadores) tiene una latencia de 4 ut , por lo que esta es la latencia total. Luego, el *scheduling* *ALAP* de la [Fig. 2b](#) muestra que los únicos operadores flexibles son M_2 y M_3 .



(a) Método *ASAP*.



(b) Método *ALAP*.

Figura 2: *Scheduling* del filtro *FIR* de 4 coeficientes.

Se calculó la movilidad de M_2 y M_3 , resultando

$$\begin{cases} m_{M2} = 1 - 0 = 1 \\ m_{M3} = 2 - 0 = 2 \end{cases} \quad (2)$$

Nótese que la movilidad de ambos multiplicadores no cambiaría si se modifica su latencia con respecto a los sumadores.

2.3. Versión Iterativa con *MAC*

Al restringir los recursos a **1 sumador y 1 multiplicador**, estamos aplicando la técnica arquitectónica de *Folding*, reutilizando el *HW* en distintos ciclos temporales.

Veamos primero como es el proceso de diseño de la máquina de estados (*FSM*): Dado que procesar una muestra requiere 4 multiplicaciones y 3 sumas, al contar con un único bloque *MAC* (Multiplicador-Acumulador), necesitamos obligatoriamente un mínimo de 4 ciclos de clock. La *FSM* contará entonces con 4 estados iterativos para computar cada salida $y[n]$:

1. Ciclo 1 (S_0): Se enruta $x[n]$ con h_0 a través de MUXes (Operación: $A_{cc} = h_0 \cdot x[n]$).
2. Ciclo 2 (S_1): Se enruta $x[n-1]$ con h_1 (Operación: $A_{cc} = A_{cc} + (h_1 \cdot x[n-1])$).

3. Ciclo 3 (S_2): Se enruta $x[n-2]$ con h_2 (Operación: $A_{cc} = A_{cc} + (h_2 \cdot x[n-2])$).
4. Ciclo 4 (S_3): Se enruta $x[n-3]$ con h_3 donde se entrega el resultado a la salida y se resetea el acumulador para la próxima muestra (Operación: $y[n] = A_{cc} + (h_3 \cdot x[n-3])$).

Este proceso se muestra gráficamente en la Fig. 3.

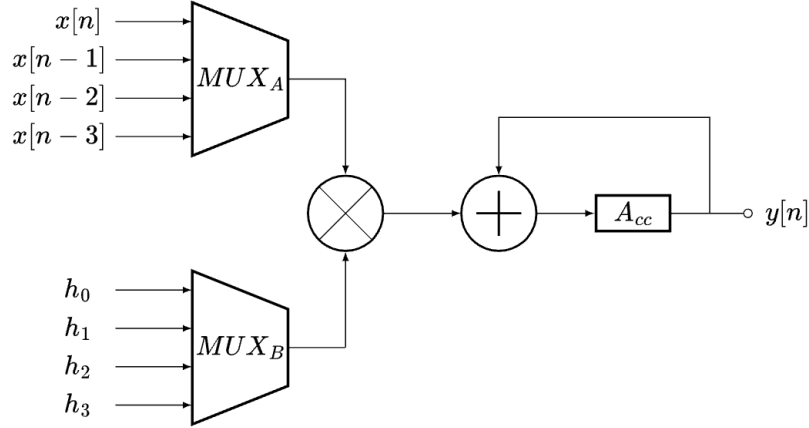


Figura 3: *FSM* de la versión iterativa del *FIR*.

Se tienen entonces que las **Estimaciones Temporales** son:

- La *Latencia* es de: 4 [ciclos] (Es el tiempo total desde que ingresa la muestra $x[n]$ hasta que se produce la salida $y[n]$).
- El *Throughput* es de $0,25 \left[\frac{\text{muestras}}{\text{ciclo}} \right]$ (se genera 1 resultado válido cada 4 ciclos de clock).

2.4. *Feed-Forward Cut-Set*

El método *Feed-Forward Cut-Set* consiste en dividir un grafo en 2 partes, trazando una línea que corta un conjunto de aristas que van a la misma dirección (no hay bucles), y luego agregando una cierta cantidad de registros en medio, con el objetivo de reducir el camino crítico.

Esta técnica se realizó sobre el filtro *FIR*, resultando en 2 etapas de *pipeline* como se muestra en la Fig. 4. Con esto, se logró un camino crítico de 2 *ut* (3 *ut* menos que sin *pipeline*) correspondiente a los multiplicadores $M_0, \dots, M_3$ y los dos sumadores S_1 y S_2 .

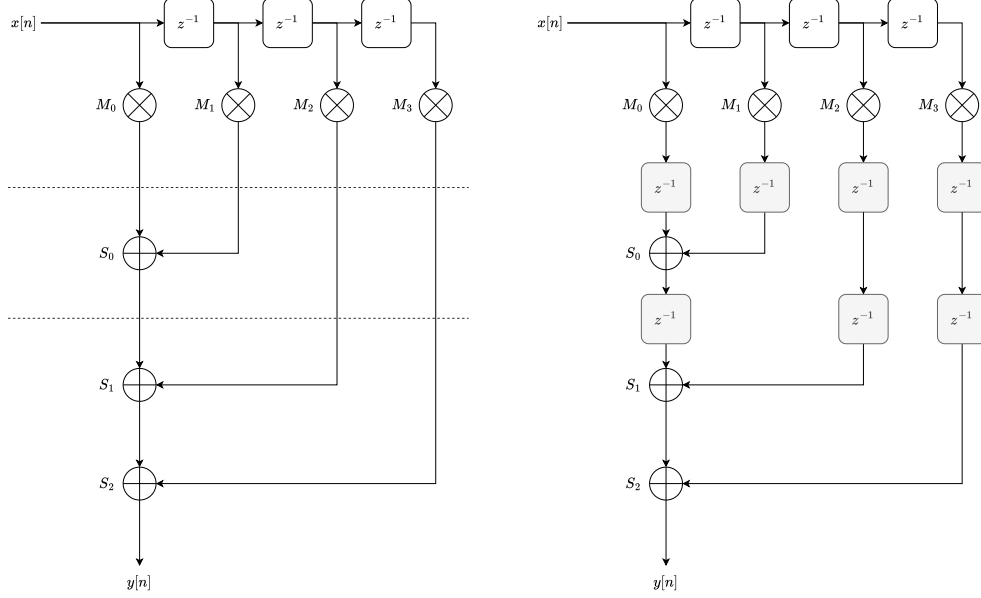


Figura 4: Filtro *FIR* con 2 etapas de *pipeline* mediante el método *Cut-Set*.

En la Tab. 1 se compara entonces la arquitectura con y sin *pipeline*, asumiendo $1 [ut] = 1 [ns]$, y se concluye que este método de *retiming* permite utilizar una frecuencia de reloj más alta para el diseño, pero a coste de aumentar la latencia, consumo y área debido al agregado de registros.

	Sin pipeline	Con pipeline
Camino crítico $[ns]$	5	2
Frecuencia máxima $[MHz]$	~ 200	~ 500
Latencia [ciclos de reloj]	0	2

Cuadro 1: Comparación entre filtro *FIR* sin pipeline y con pipeline.

2.5. Comparación del *PPA*

Para este análisis *PPA* (Power, Performance, Area) contrastamos la **Versión Iterativa** (véase la Subsección 2.3) contra la **Versión Pipeline** (véase la Subsección 2.4) insertando etapas de registros para obtener la Tab. 2.

Métrica PPA	Versión Iterativa (Folding)	Versión Pipeline (Unfolded)
Latencia	4 ciclos	2 ciclos (según Tab. 1)
Throughput	0.25 muestras/ciclo (Bajo)	1 muestra/ciclo (Alto a ~ 500 MHz)
Área Estimada	Baja: 1 Mul, 1 Add, MUXes y lógica FSM.	Alta: 4 Mul, 3 Add, y abundantes registros (z^{-1}) por el <i>cut-set</i> .
Consumo	Bajo: Ideal para bajo consumo; menor conmutación.	Alto: Gran cantidad de hardware combinacional y registros activos.

Cuadro 2: Comparativa de recursos y métricas de tiempo (PPA) entre arquitecturas *FIR*.

Se tiene que la planificación define el consumo y área sin tocar la matemática subyacente del algoritmo. La arquitectura iterativa aplica multiplexación en el tiempo, ahorrando área de silicio y reduciendo la potencia dinámica, lo que la hace idónea para sistemas embebidos de bajo consumo y baja velocidad. Por el contrario, la versión con *pipeline* disminuye el camino crítico de 5 $[ns]$ a 2 $[ns]$, mejorando sustancialmente el paralelismo y garantizando 1 resultado por ciclo a una frecuencia máxima mucho mayor, a expensas de un incremento en la latencia inicial, área y consumo.

2.6. Límite de Iteración (*IPB*)

El *límite de iteración* (*IPB*) es una métrica utilizada para obtener el mínimo período alcanzable en un diseño, y para obtenerlo se debe realizar lo siguiente:

1. Identificar los lazos, es decir, los caminos del *DFG* que salen de un nodo y vuelven al mismo nodo, recorriéndolo en sentido dirigido.
2. Sumar las latencias de todos los nodos del lazo (T).
3. Contar la cantidad de registros sobre ese lazo (D).
4. Calcular el *límite de lazo* para cada lazo i como $LB_i = T_i/D_i$.
5. El LB_i más alto es el *IPB*.

Se puso en práctica este método convirtiendo el filtro *FIR* en un filtro *IIR* de 1^{er} orden, utilizando la aproximación de Padé. Mientras que el *FIR* tiene la forma que se muestra en la [Ec. 1](#), se tiene que el filtro *IIR* termina siendo:

$$y[n] = \frac{h_2}{h_1}y[n-1] + h_0x[n] + \left(h_1 - \frac{h_0h_2}{h_1}\right)x[n-1] \quad (3)$$

Lo que se puede reescribir simplemente como muestra la [Ec. 4](#).

$$y[n] = a_1y[n-1] + b_0x[n] + b_1x[n-1] \quad (4)$$

La arquitectura de este *IIR* se muestra en la [Fig. 5](#). Teniendo en cuenta que solo existe un nodo en este caso, se obtiene que:

$$IPB = \frac{T_0}{D_0} = \frac{3}{1} = 3 [ut] \quad (5)$$

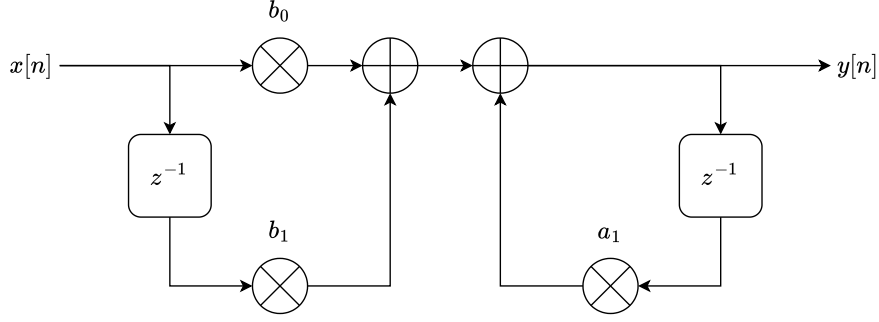


Figura 5: Esquema del filtro *IIR*.

Se puede observar que, de todas formas, el camino crítico viene dado por el multiplicador b_1 y los dos sumadores, pero sobre él se pueden aplicar las técnicas de *pipeline* vistas anteriormente. Luego, suponiendo que el camino crítico está en el lazo, se puede aplicar el método *C-Slow*, que consiste en reemplazar todos los registros del *DFG* por C registros. En la Fig. 6 se puede ver este método aplicado al *IIR* con $C = 2$, donde se agregó un registro más al lazo de realimentación y uno al lazo *feed-forward*; a este último se le tuvo que agregar otro registro más para mantener la función de transferencia.

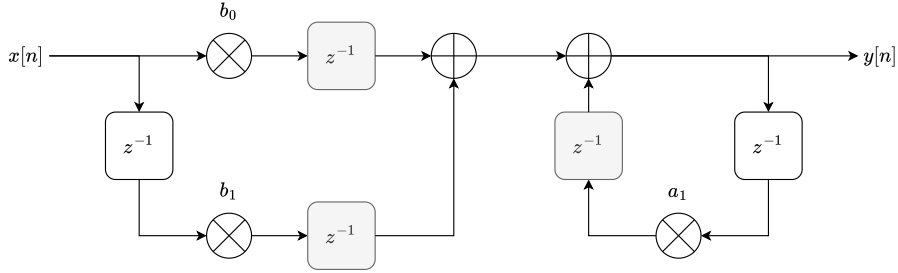


Figura 6: *C-Slow* aplicado al filtro *IIR*.

Con esto, se obtiene un *IPB* 2 veces menor:

$$IPB = \frac{3}{2} = 1,5 [ut] \quad (6)$$

3. Conclusiones

El presente informe demostró cómo la planificación temporal y las transformaciones arquitectónicas impactan directamente en las métricas *PPA* de un sistema *DSP*, sin alterar su matemática subyacente.

Por un lado, se comprobó que la técnica de *Folding* (arquitectura iterativa) minimiza drásticamente el área (empleando un solo bloque *MAC*) y el consumo a costa de reducir el *throughput*, resultando idónea para sistemas embebidos estrictos. En contraste, el *Pipe-*

lining (mediante *Cut-Set*) permitió elevar la frecuencia máxima a $500 [MHz]$ y garantizar 1 resultado por ciclo, a expensas de un incremento en área y latencia inicial.

Finalmente, el análisis de sistemas *IIR* evidenció la restricción impuesta por el límite de iteración (*IPB*) en lazos realimentados, el cual logró reducirse a la mitad mediante la técnica *C-Slow*. En definitiva, el diseño de arquitecturas *DSP* no admite una única solución, sino que exige un compromiso de diseño: ya sea priorizar rendimiento (*Pipeline/C-Slow*) o priorizar recursos (*Folding*) según los requisitos de cada aplicación.

4. Referencias

- [1] I. Amulya, *Implementation and Design of FIR Filters using Verilog HDL and FPGA*, Academia.edu, 2020. dirección: <https://www.academia.edu/figures/5840702/figure-2-in-case-of-tap-filter-the-filter-co-efficient-hg-is>
- [2] E. Monja y G. R. D. Zúñiga, *Módulo 6 - Ejercicios*, Repositorio en GitHub, Diseño Digital para Sistemas DSP, Archivos desarrollados para los ejercicios del Módulo N°6, 2026. dirección: https://github.com/ErnestMonja/Diseno_Digital_para_Sistemas_DSP/tree/main/M%C3%B3dulo%206%20-%20Ejercicios